

```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)
    stop("pH_super approximation relative error exceeds 10%")
  z1_frac <- z1_q
}

```

```

    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria
hr.threshold <- 1.25 # Initial candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

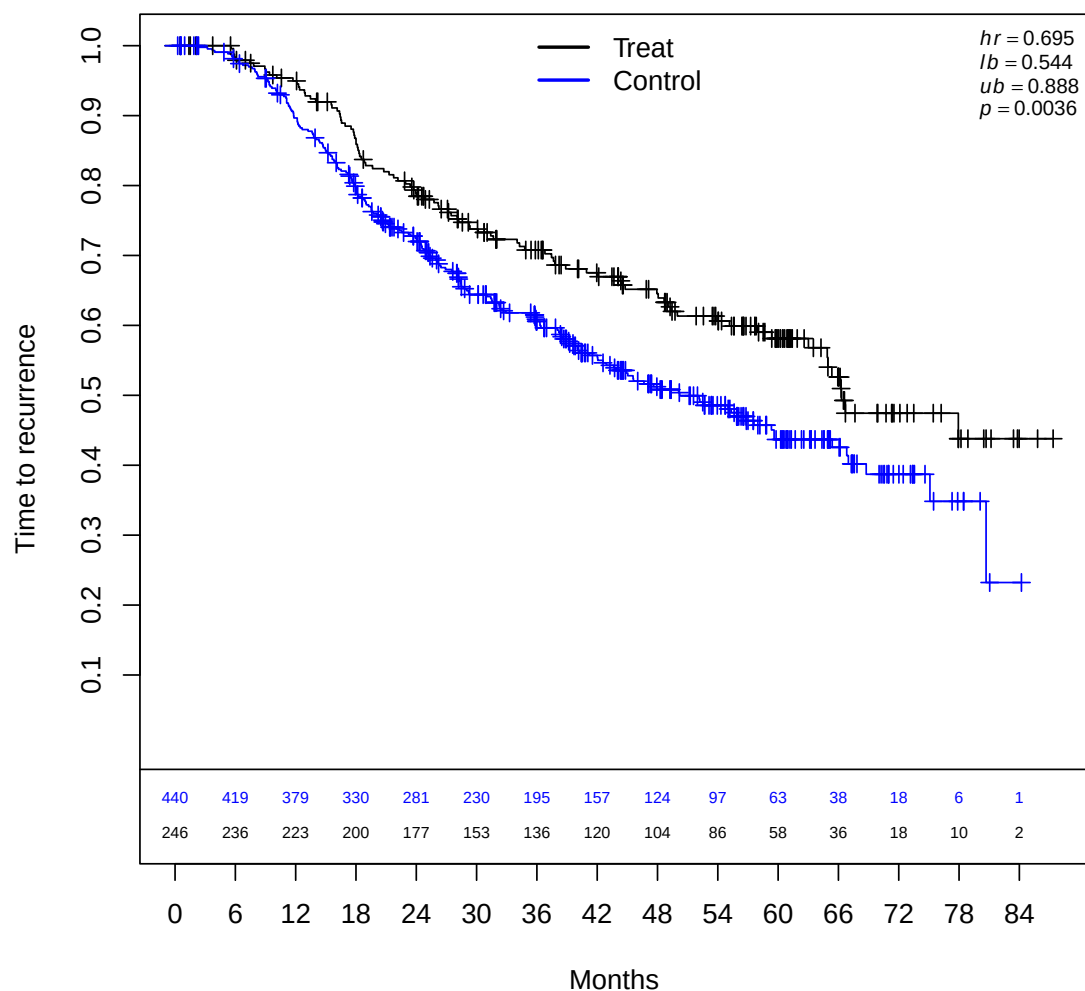
maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

mod.harm <- "alt"
hrH.target <- 2.5
# out.loc = NULL turns off file creation
N <- 700
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)

```



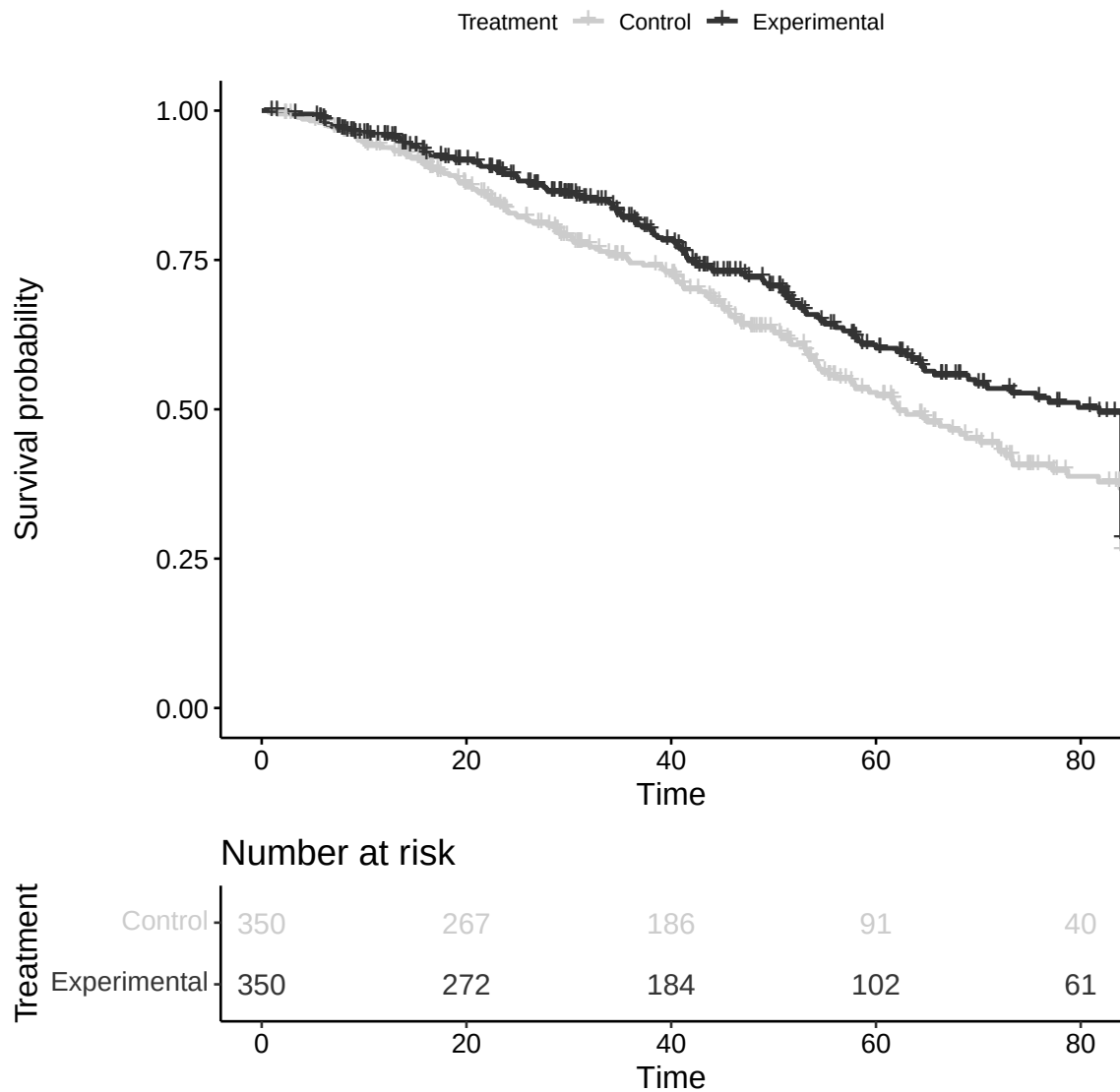
```
## Super-population empirical harm and non-harm hazard ratios= 2.499996 0.6466405
## Causal HR (empirical ITT)= 0.7125876
```

```
n_add_noise <- 3
sim <- 1

dgm <- this.dgm$dgm
x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
  legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
  risk.table = TRUE, risk.table.col = "strata")
```



```

if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
  # Add 5 noise
  x$noise1 <- rnorm(N)
  x$noise2 <- rnorm(N)
  x$noise3 <- rnorm(N)
  x$noise4 <- rnorm(N)
  x$noise5 <- rnorm(N)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
    "noise2", "noise3", "noise4", "noise5")
}

if (n_add_noise == 3) {

```

```

set.seed(8316951 + 1000 * sim)
# Add 3 noise
x$noise1 <- rnorm(N)
x$noise2 <- rnorm(N)
x$noise3 <- rnorm(N)
confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
                     "noise2", "noise3")
}

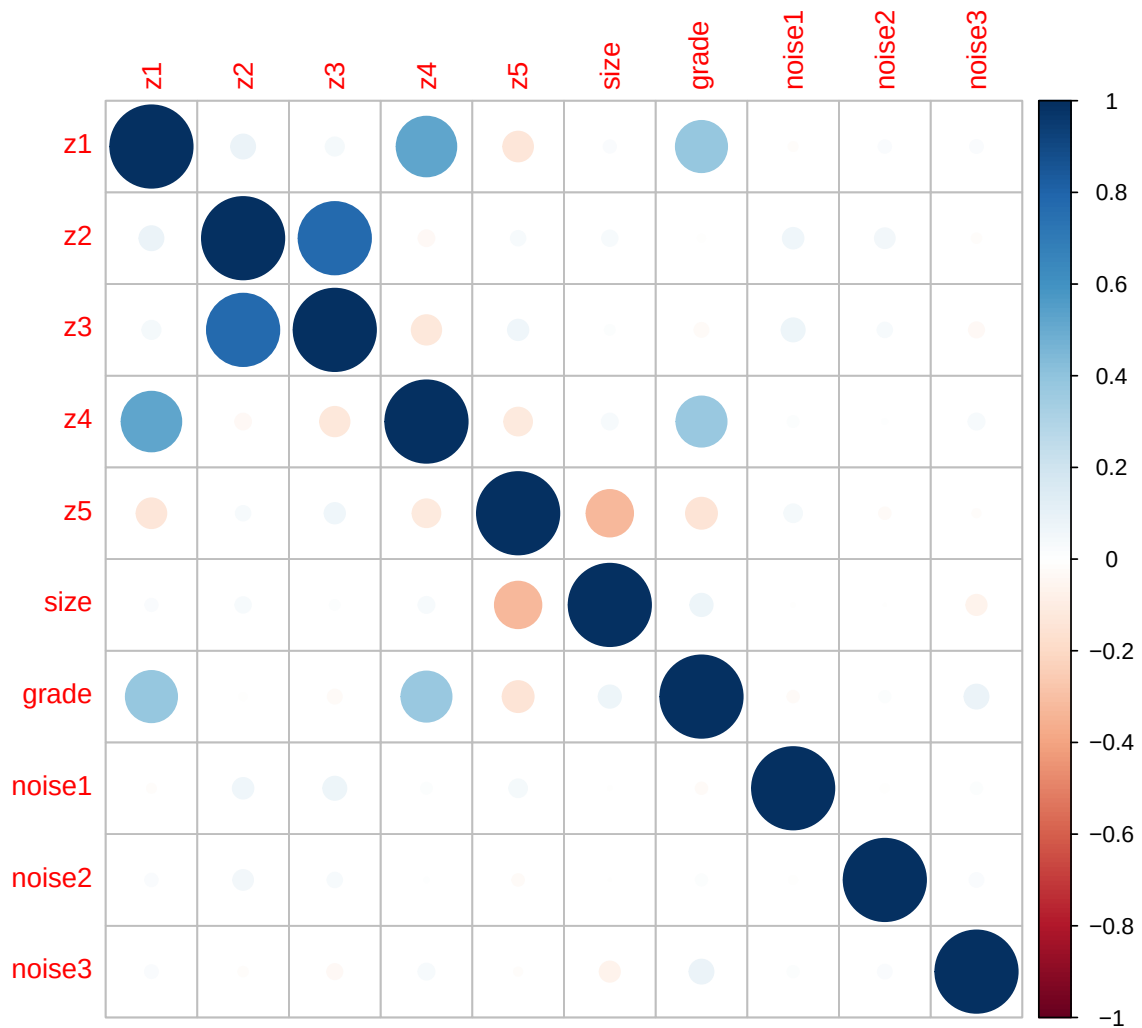
# More challenging which_replace <- which(confounders.name=='z1')
# confounders.name[which_replace] <- 'er'

cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5+size+grade+noise1+
coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef) se(coef)      z      p
## treat  -0.145329  0.864738  0.107130 -1.357  0.17492
## v11      0.703034  2.019872  0.140260  5.012 5.38e-07
## v21     -0.642606  0.525920  0.198522 -3.237  0.00121
## v31      0.942966  2.567586  0.201563  4.678 2.89e-06
## v41      0.585870  1.796553  0.135123  4.336 1.45e-05
## v51     -0.809896  0.444904  0.112442 -7.203 5.90e-13
## size      0.003725  1.003732  0.003547  1.050  0.29360
## grade   -0.047301  0.953800  0.100796 -0.469  0.63887
## noise1 -0.043123  0.957793  0.054739 -0.788  0.43082
## noise2  0.133276  1.142565  0.054121  2.463  0.01379
## noise3  0.031278  1.031772  0.052261  0.598  0.54951
##
## Likelihood ratio test=193.1 on 11 df, p=< 2.2e-16
## n= 700, number of events= 366

Zm <- cor(x[, c(confounders.name)])
corrplot(Zm)

```



```
# Options Allconfounders.name is list of confounders within analysis dataset
# (1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
# Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
# Lasso used to reduce dimension Continuous covariates are cut at medians
# However, if GRF selects a covariate cut then only that cut is used: For
# example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
# <= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
# use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
# F & use_grf =T Median cuts (unless selected via GRF) as in (2) However no
# possible dimension reduction via lasso All categorical factors included

use_lasso <- TRUE
use_grf <- FALSE
use_grf_only <- FALSE

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 12,
  frac.tau = 1, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
```

```

event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
pstop_futile = pstop_futile, pconsistency.threshold = pconsistency.threshold,
stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
plot.sg = TRUE, vi.grf.min = vi.grf.min)

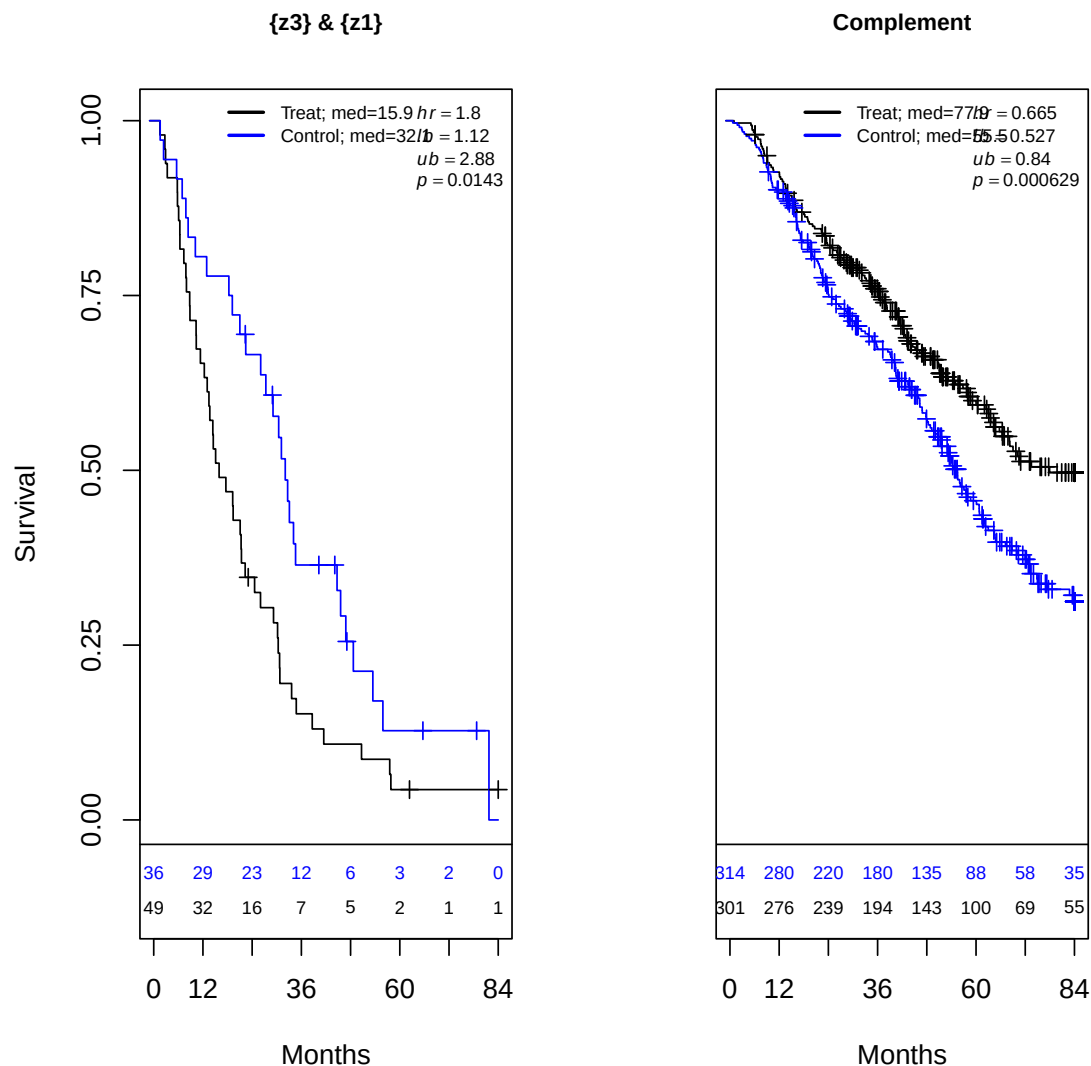
## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade
## 10 x 1 sparse Matrix of class "dgCMatrix"
##          s0
## z1      0.711838218
## z2     -0.587450265
## z3      0.896060152
## z4      0.570370503
## z5     -0.812387264
## size    0.002954375
## grade  -0.045566077
## noise1 -0.042282250
## noise2  0.129497580
## noise3  0.028581145
## Cox-LASSO selected: z1 z2 z3 z4 z5 size grade noise1 noise2 noise3
## Cox-LASSO not selected:
## Median cuts (after Lasso if use_lasso) size noise1 noise2 noise3
## Categorical (after Lasso if use_lasso) z1 z2 z3 z4 z5 grade
## Prior to removing (if in GRF): size noise1 noise2 noise3
## Left (to cut at median) after removing others if in GRF: size noise1 noise2 noise3
## # of candidate subgroup factors= 10
## [1] "size <= median(size)"      "noise1 <= median(noise1)"
## [3] "noise2 <= median(noise2)"  "noise3 <= median(noise3)"
## [5] "z1"                        "z2"
## [7] "z3"                        "z4"
## [9] "z5"                        "grade"
## Confounders per grf screening q6 q7 q5 q10 q4 q9 q1 q3 q2 q8
##   FSconfounders.name      vi.cs  vi_ratio
## 6                q6 0.27747144 2.7747144
## 7                q7 0.14768052 1.4768052
## 5                q5 0.10312434 1.0312434
## 10               q10 0.09385928 0.9385928
## 4                q4 0.08784151 0.8784151
## 9                q9 0.06167906 0.6167906
## 1               q1 0.06044896 0.6044896
## 3                q3 0.06030568 0.6030568
## 2                q2 0.05396838 0.5396838
## 8                q8 0.05362084 0.5362084
## Number of unique levels (L) and possible subgroups= 21 2097151
## Number of possible subgroups (in millions)= 2.097151
## # of subgroups based on # variables > k.max and excluded (per million) 2.09692
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 26 22
## # of subgroups with sample size less than criteria 30
## # of subgroups meeting all criteria = 201
## # of subgroups fitted (Cox model estimable) = 201
## *Subgroup Searching Minutes=* 0.1780667
## Number of subgroups meeting HR threshold 14

```

```

## # of candidate subgroups (meeting HR criteria) = 14
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K    n    E d1    m1    m0    HR L(HR) U(HR) q6.0 q6.1 q7.0 q7.1 q5.0 q5.1
## 1: 2 85 75 46 15.94 32.07 1.80 1.12 2.88 0 0 0 1 0 1
## 2: 2 84 67 30 22.30 39.47 1.74 1.06 2.85 0 0 0 0 0 1
## 3: 2 109 89 53 20.22 34.58 1.70 1.11 2.61 0 1 0 0 0 1
## 4: 2 87 36 19 63.58 NA 1.61 0.83 3.11 0 0 0 0 1 0
## 5: 2 77 30 15 64.35 NA 1.61 0.78 3.30 0 0 0 0 0 0
## 6: 2 63 22 12 68.24 NA 1.57 0.67 3.65 0 0 0 0 0 0
## 7: 1 89 37 19 63.58 NA 1.50 0.78 2.87 0 0 0 0 0 0
## 8: 2 118 88 50 21.19 30.47 1.49 0.98 2.28 0 0 0 1 0 0
## 9: 2 95 76 38 20.32 40.31 1.48 0.94 2.32 0 0 0 0 0 1
## 10: 2 186 105 58 41.44 55.88 1.42 0.97 2.09 0 1 0 0 0 0
##      q10.1 q10.2 q10.3 q4.0 q4.1 q9.0 q9.1 q1.0 q1.1 q3.0 q3.1 q2.0 q2.1 q8.0
## 1: 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## 2: 0 0 0 0 0 0 0 0 0 0 0 0 1 0
## 3: 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## 4: 1 0 0 0 0 0 0 0 0 0 0 0 0 0
## 5: 1 0 0 0 0 0 0 0 0 0 0 0 0 1
## 6: 1 0 0 0 0 0 1 0 0 0 0 0 0 0
## 7: 1 0 0 0 0 0 0 0 0 0 0 0 0 0
## 8: 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## 9: 0 0 0 1 0 0 0 0 0 0 0 0 0 0
## 10: 0 0 0 0 0 0 0 1 0 0 0 0 0 0
##      q8.1
## 1: 0
## 2: 0
## 3: 0
## 4: 0
## 5: 0
## 6: 0
## 7: 0
## 8: 1
## 9: 0
## 10: 0
## Consistency 0.99
## # of splits= 400
## Model, % Consistency Met= {z3} {z1} 0.99
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K M.1 M.2
## 1: 0.99 85 1 1 2 {z3} {z1}

```



```
## [1] "{z3}" "{z1}"
## % consistency criteria met= 0.99
## SG focus= hr
## Subgroup Consistency Minutes= 0.0229
## Subgroup found
## Minutes overall= 0.2197

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, details = TRUE)

## tau, maxdepth= 77.92977 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2  -10.436478   325.000000   2.821312   10.436478   325.000000
## 2         3    3.725444   375.000000   2.511110   -3.725444   375.000000
## 3         4    4.034948   136.000000   4.137325   -4.034948   136.000000
## 4         5  -12.019252    94.000000   4.954000   12.019252    94.000000
## 5         6  -15.525492   217.000000   3.448402   15.525492   217.000000
## 6         7    7.728358   253.000000   3.051644   -7.728358   253.000000
```

```
##   treated.se      diff Nsg depth
## 1   2.821312 -20.872956 325     1
## 2   2.511110   7.450887 375     1
## 3   4.137325   8.069896 136     2
## 4   4.954000 -24.038505  94     2
## 5   3.448402 -31.050984 217     2
## 6   3.051644  15.456716 253     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 6         7      7.728358   253.000000   3.051644   -7.728358   253.000000
##   treated.se      diff Nsg depth
## 6   3.051644 15.45672 253     2
## [1] "size <= 22"      "noise3 <= 0.34" "z2 <= 0"
## [1] "z2 <= 0"
```

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 20

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_mat")

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")

## Minutes for Boots 20 0.2486667

cat("Projection per 100", c(tB.min * (100/NB)), "\n")
```

```

## Projection per 100 1.243333

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")

## Propn bootstrap subgroups found = 1

# How many timmed out
cat("Number timmed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmns_search > max.minutes)),
    "\n")

## Number timmed out= 0

H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_1,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0.05)

Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_1,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0.05)

print(H_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 1.800236 0.4319659 1.124822 2.88121 1.449638 0.738049 0.5344297 3.932138

print(Hc_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower
## 1: 0.6653353 0.07929625 0.5267352 0.8404053 0.6549832 0.08849245 0.5026056
##           H1_upper
## 1: 0.8535579

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")

## Overall minutes for analysis 0.5339667

save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, file = "output/sim-FS4_grfOnly_ex1.Rdata")

## Confounders evaluated in Forestsearch:
## [1] "size <= median(size)"      "noise1 <= median(noise1)"
## [3] "noise2 <= median(noise2)"  "noise3 <= median(noise3)"
## [5] "z1"                        "z2"
## [7] "z3"                        "z4"
## [9] "z5"                        "grade"

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT					
ITT	0.799	0.65	0.982	700	.
H subgroup estimates					
H true	1.8	1.125	2.881	85	.
$\hat{H}$	1.8	1.125	2.881	85	85
$\hat{H}_{bc}$	1.45	0.535	3.932	85	85
H-complement subgroup estimates					
H^c true	0.665	0.527	0.839	615	.
$\hat{H}^c$	0.665	0.527	0.839	615	615
$\hat{H}_{bc}^c$	0.655	0.503	0.852	615	615